

admin_dashboard.js - Complete Documentation

File Location: src/js/admin_dashboard.js
Purpose: Admin dashboard interactivity for managing all admin functions
Dependencies: modal1.js, history_handler.js
Used by: admin_homepage.html

Overview

The admin_dashboard.js file implements the complete admin management interface. It handles opening list modals, loading data from various admin endpoints, and rendering different types of lists (events, users, complaints, feedback, items, requests). The file uses a generic modal system with type-based data loading.

Module Variables

currentListType

```
let currentListType = '';
```

Purpose: Track which list is currently displayed in modal.

currentListData

```
let currentListData = [];
```

Purpose: Store the fetched list data for rendering.

Core Functions

openAdminListModal(type)

Purpose: Opens generic list modal for different admin functions.

Parameters:

- type (string) - List type to display. Possible values:
 - 'edit_event' - Events (editable)
 - 'total_events' - All events (view-only)
 - 'active_users' - Active users (view)
 - 'edit_users' - Users (editable)
 - 'complaints' - Event complaints
 - 'feedback' - Event feedback
 - 'items_reported' - Lost/found items
 - 'event_requests' - Student event requests

Returns: void

What it does:

- Sets currentListType to the requested type
- Opens admin_list_modal.html
- Calls loadListData(type) to fetch and display data

Code:

```
function openAdminListModal(type) {
    currentListType = type;

    openModal('../frontends/admin/admin_list_modal.html', 'adminListModal', function(modal) {
        loadListData(type);
    });
}
```

Example Usage:

```
// User clicks "View All Events"
openAdminListModal('total_events');

// User clicks "Edit Users"
openAdminListModal('edit_users');

// User clicks "View Complaints"
openAdminListModal('complaints');
```

loadListData(type)

Purpose: Fetches appropriate data based on list type.

Parameters:

- `type` (string) - List type to load

Returns: void (async)

What it does:

1. Shows loading message
2. Maps type to appropriate PHP endpoint
3. Fetches data from endpoint
4. Stores in `currentListData`
5. Calls `renderAdminList()`

Endpoint Mapping:

```
switch(type) {
  case 'edit_event':
  case 'total_events':
    url = '../php/admin/get_all_events.php';
    break;
  case 'active_users':
  case 'edit_users':
    url = '../php/admin/get_all_users.php';
    break;
  case 'complaints':
  case 'feedback':
    url = '../php/admin/get_all_events.php';
    break;
  case 'items_reported':
    url = '../php/admin/get_reported_items.php';
    break;
  case 'event_requests':
    url = '../php/admin/get_event_requests.php';
    break;
}
```

Code:

```

async function loadListData(type) {
  const contentDiv = document.getElementById('adminListContent');
  if (!contentDiv) return;

  contentDiv.innerHTML = '<p style="text-align:center; padding:40px;">Loading data...</p>';

  let url = '';
  // ... determine URL from type

  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error('Server error');

    const data = await res.json();

    currentListData = Array.isArray(data) ? data : [];

    if (currentListData.length === 0) {
      contentDiv.innerHTML = '<p style="text-align:center; padding:40px;">No records found.</p>';
    } else {
      renderAdminList(currentListData, type);
    }
  } catch (err) {
    console.error(err);
    contentDiv.innerHTML = `<p style="color:red; text-align:center;">Failed to load data.<br><small>${err.message}</small></p>`;
  }
}

```

renderAdminList(data, type)

Purpose: Renders appropriate HTML for different list types.

Parameters:

- data (array) - Array of items to render
- type (string) - List type (determines rendering)

Returns: void

What it does:

1. Sets modal title based on type
2. Renders appropriate HTML structure
3. Creates list items with specific actions
4. Shows view/edit buttons

Title Mapping:

```

const titles = {
  'edit_event': 'Edit Events',
  'total_events': 'All Events',
  'active_users': 'Active Users',
  'edit_users': 'Edit Users',
  'complaints': 'Pending Complaints',
  'feedback': 'All Feedback',
  'items_reported': 'Reported Items',
  'event_requests': 'Event Requests'
};

```

Rendering by Type:

Events

```
case 'edit_event':
case 'total_events':
  infoHTML = `
    <div class="list-info">
      <h4>${item.ename}</h4>
      <p>${item.date} | ${item.category} | ${item.venue}</p>
    </div>`;
  buttonText = type === 'edit_event' ? 'Edit' : 'View';
  onclick = `viewEventInList(${item.event_id}, '${type}')`;
  break;
```

Rendered Output:

Sports Day 2024
2024-05-15 sports Ground
[Edit]

Users

```
case 'active_users':
case 'edit_users':
  infoHTML = `
    <div class="list-info">
      <h4>${item.uname}</h4>
      <p>${item.email} | ${item.role}</p>
    </div>`;
  buttonText = type === 'edit_users' ? 'Edit' : 'View';
  onclick = `viewUserInList(${item.user_id}, '${type}')`;
  break;
```

Complaints

```
case 'complaints':
  const compCount = parseInt(item.complaint_count) || 0;
  infoHTML = `
    <div class="list-info">
      <h4>${item.ename}</h4>
      <p>${compCount} complaint(s) | ${item.date}</p>
    </div>`;
  buttonText = compCount > 0 ? 'View Complaints' : 'No Complaints';
  onclick = compCount > 0 ? `viewEventComplaints(${item.event_id})` : `alert('No complaints for this event yet.')`;
  break;
```

Reported Items

```
case 'items_reported':
  const itemType = item.item_type === 'lost' ? '🔍 Lost' : '📍 Found';
  infoHTML = `
    <div class="list-info">
      <h4>${item.item_name}</h4>
      <p><strong>${itemType}</strong> • ${item.report_date} | ${item.location}</p>
    </div>`;
  buttonText = 'View Details';
  onclick = `viewReportedItem(${item.id}, '${item.item_type}')`;
  break;
```

Admin Action Functions

viewEventInList(eventId, type)

Purpose: Opens event details or editing modal.

Parameters:

- `eventId` (integer) - Event to view/edit
- `type` (string) - 'edit_event' or 'view'

If 'edit_event':

- Opens `edit_event.html` modal
- Pre-populates with event data

If 'view':

- Opens event details (read-only)
-

viewEventComplaints(eventId)

Purpose: Shows all complaints for specific event.

Parameters:

- `eventId` (integer) - Event to view complaints for

What it does:

1. Fetches complaints from `get_event_complaints.php`
 2. Displays complaint list
 3. Each complaint has "View" button
 4. Opens complaint detail modal on click
-

viewEventFeedback(eventId)

Purpose: Shows all feedback for specific event.

Parameters:

- `eventId` (integer) - Event to view feedback for

What it does:

1. Fetches feedback from `get_event_feedback.php`
 2. Shows feedback with ratings (1-5 stars)
 3. Displays feedback text
-

viewUserInList(userId, type)

Purpose: Opens user details or edit modal.

Parameters:

- `userId` (integer) - User to view/edit
 - `type` (string) - 'edit_users' or 'view'
-

viewReportedItem(itemId, itemType)

Purpose: Shows details of lost/found item.

Parameters:

- `itemId` (integer) - Item ID
- `itemType` (string) - 'lost' or 'found'

What it does:

1. Fetches item details
 2. Shows item photo, description, location
 3. May show action buttons (mark as found, etc.)
-

viewEventRequest(requestId)

Purpose: Shows student-requested event details.

Parameters:

- requestId (integer) - Request to review

What it does:

- Fetches request details
- Shows proposed event information
- Shows student who requested it
- Approve/Reject buttons

Data Flow Example

Complete Workflow: Viewing Complaints



Integration Points

With modal1.js

```
// Uses generic modal system
openModal('../frontends/admin/admin_list_modal.html', 'adminListModal', function() {
  loadListData(type);
});
```

With history_handler.js

```
// Admin can view system history
openHistoryModal('eventRequest');
```

Modal Structure

admin_list_modal.html Expected Structure

```
<div class="modal-content">
  <div class="modal-header">
    <h2 id="listModalTitle">Admin List</h2>
    <span class="close" onclick="closeModal('adminListModal')">×</span>
  </div>

  <div id="adminListContent">
    <!-- List items rendered here -->
  </div>
</div>
```

Generated List Item Structure

```
<div class="list-item">
  <div class="list-info">
    <h4>Item Title</h4>
    <p>Metadata info</p>
  </div>
  <button class="action-btn" onclick="...">View/Edit</button>
</div>
```

Error States

No Records Found

<p style="text-align:center; padding:40px;">No records found.</p>

Load Error

<p style="color:red; text-align:center;">Failed to load data.
<small>Error message</small></p>

Loading State

<p style="text-align:center; padding:40px;">Loading data...</p>

Performance Considerations

✅ Current Strengths

- Lazy loading (data fetched on demand)
- Efficient rendering with `.join('')`
- Reusable modal pattern

⚠️ Potential Improvements

1. Pagination

```
// Add pagination to large lists
async function loadListData(type, page = 1) {
  const url = buildUrl(type, { page, limit: 20 });
  // ...
}
```

2. Search/Filter

```
function filterListData(searchTerm) {
  const filtered = currentListData.filter(item =>
    item.ename?.includes(searchTerm) ||
    item.uname?.includes(searchTerm)
  );
  renderAdminList(filtered, currentListType);
}
```

3. Sort Options

```
function sortListData(sortBy) {
  currentListData.sort((a, b) => {
    if (sortBy === 'date') return new Date(a.date) - new Date(b.date);
    if (sortBy === 'name') return a.ename.localeCompare(b.ename);
    return 0;
  });
  renderAdminList(currentListData, currentListType);
}
```

Summary

admin_dashboard.js provides:

Feature	Purpose
openAdminListModal()	Open list modal for any type
loadListData()	Fetch data from PHP
renderAdminList()	Render type-specific HTML
Action Functions	Handle view/edit operations

Creates a flexible admin interface for managing all system resources.

Last Updated: May 22, 2026

File Size: ~200 lines of code

Complexity: Medium-High

Criticality: Very High (admin interface)